URSim fault catalogue

Every software fault we can provoke on the Universal Robots simulator, and what it really produces

URSim 5.26.1.140514 (UR3e) · generated 2026-10-29 · 5 captures per fault

Method. Each fault is provoked by `tools/fault_injection/inject.py` and captured by `tools/fault_injection/capture.py`: the robot is brought to RUNNING, our three live streams are started (signals of the Real-Time interface at 500 Hz, fault codes of the Primary interface, the three log files of the container), the fault is injected, the simulator is observed for 15 s, then the fault is reverted and the time to RUNNING is measured. The raw evidence of every run is kept in `data/catalogue/`.

No guessing. Columns 3 and 4 are computed by `catalogue.py` from the captures only: each fact gives the number of runs where it was seen (e.g. 5/5). Log lines are shown as parsed by the agent's own connector, with numbers replaced by #. The official meaning of a fault code comes from the Universal Robots error code pages (software 5.26). Columns 1, 2 and 5 are written by hand in `catalogue.yaml`: commands copied from the injection script, and official sources, each with what it supports. Facts observed outside the dataset are marked as notes, with how they were observed.

Fault	Category	Fault code	Warning/error in a log	Visible in the signals	Black box
`wrong_payload`	configuration	✗	✗	✗	✗
`script_error`	program	✗	✓	✗	✗
`protective_stop`	safety	✓	✓	✓	✓
`controller_crash`	process	✗	✓	✓	✗
`freeze`	process	✗	✗	✓	✗
`polyscope_freeze`	process	✗	✓	✗	✗
`network_cut`	network	✗	✓	✓	✓
`cpu_starvation`	resource	✗	✗	✓	✗
`cpu_overload`	resource	✗	✗	✓	✗
`network_delay`	network	✗	✓	✓	✗
`packet_loss`	network	✗	✓	✓	✗
`port_blocked`	network	✗	✓	✓	✗
`disk_full`	resource	✗ (probe: ✓)	✓	✗ (probe: ✓)	✗ (probe: unreadable)

"Visible in the signals": a stream cut, a change of robot/safety mode, or a median rate below 450 Hz, i.e. more than 6 % below the 481 Hz received when nothing disturbs the stream (500 Hz by design). "Probe": `disk_full` is silent at rest, so its captures send `protective_stop()` 2 s after the fill to force the controller to write; the code and the black box then come from that protective stop. A fault with only ✗ in the first two columns cannot raise an alert from logs and fault codes alone: it needs the rules on the signals (summary.md, section 3).

Fault	How to provoke / revert	Logs and fault codes (observed)	Effect on signals (observed)	Category, sources and proof
wrong_payload Payload declared as 2 kg while the tool is empty: the torques no longer match.	*Provoke:* URScript on port 30002: set_target_payload(2.0, [0, 0, 0.05]) *Revert:* URScript on port 30002: set_target_payload(0.0, [0, 0, 0])	**No fault code** controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_STOPPING to PROGRAM_STATE_STOPPED` controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_STOPPED to PROGRAM_STATE_RUNNING` controller INFO (5/5): `Starting program: set_target_payload` controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_RUNNING to PROGRAM_STATE_STOPPING`	Samples received: 481 per second over the 17 s window (median; 500 by design). No stream cut (0/5). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Back to RUNNING: 5/5, 1.1 s (median).	**configuration** https://www.universal-robots.com/manuals/EN/HTML/SW5_26/Content/prod-scriptmanual/all_scripts/set_target_payload.htm: set_target_payload(m, cog, ...) sets the mass and centre of gravity of the active payload. Evidence: `data/catalogue/wrong_payload-*.json` (5 runs, 2026-10-28 23:31:20 to 23:59:09). *Note: Not verified yet: the torque/current mismatch is expected only while the robot moves (the captures are done with the robot idle).*
script_error Invalid URScript program sent to the controller.	*Provoke:* URScript on port 30002: movej(get_actual_joint_positions(), a=1.0, v=) (missing value) *Revert:* nothing to undo (the program is rejected)	**No fault code** controller ERROR (5/5): `TCPReceiver::parseLine: Parse Error: movej(get_actual_joint_positions(), a=#, v=)` controller ERROR (5/5): `RuntimeException: Syntax error on line #: movej(get_actual_joint_positions(), a=#, v=)`	Samples received: 481 per second over the 17 s window (median; 500 by design). No stream cut (0/5). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Back to RUNNING: 5/5, 1.1 s (median).	**program** https://www.universal-robots.com/articles/ur/interface-communication/overview-of-client-interfaces/: Ports 30001/30002 accept URScript commands. Evidence: `data/catalogue/script_error-*.json` (5 runs, 2026-10-28 23:31:46 to 23:59:35).
protective_stop Protective stop: the robot stops and refuses to move.	*Provoke:* URScript on port 30002: protective_stop() *Revert:* Dashboard (port 29999): unlock protective stop, power on, brake release	**C209A0** (5/5 runs, after 24 ms) *Official: C209 A protective stop was triggered (for test purposes only)* polyscope ERROR (3/5): `File attachment not found: /tmp/log/urcontrol/current` pendant WARNING (5/5): `C209A0` controller WARNING (5/5): `Protective stop. Error code: C209A0` controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_STOPPING to PROGRAM_STATE_STOPPED` controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_STOPPED to PROGRAM_STATE_RUNNING` controller INFO (5/5): `Starting program: protective_stop` controller INFO (5/5): `Runtime state change: from PROGRAM_STATE_RUNNING to PROGRAM_STATE_STOPPING`	Samples received: 481 per second over the 17 s window (median; 500 by design). No stream cut (0/5). safety_mode: PROTECTIVE_STOP (5/5) Dashboard after 15 s: RUNNING / PROTECTIVE_STOP. Black box: 5/5 runs (CONTROLLER_REPORTED_ERROR ×5). Back to RUNNING: 5/5, 1.1 s (median).	**safety** https://www.universal-robots.com/manuals/EN/HTML/SW5_26/Content/prod-scriptmanual/all_scripts/protective_stop.htm: protective_stop() triggers a protective stop, pausing the program and stopping motion. https://docs.universal-robots.com/tutorials/communication-protocol-tutorials/rtde-guide.html: safety_status value 3 = PROTECTIVE_STOP (the Real-Time 'safety_mode' channel is listed there as deprecated, without values). https://www.universal-robots.com/articles/ur/dashboard-server-e-series-port-29999/: "Unlock Protective Stop" Dashboard command. https://www.universal-robots.com/manuals/EN/HTML/SW5_26/Content/prod-err-codes/topics/CODE_209.html: official page of the observed code. Evidence: `data/catalogue/protective_stop-*.json` (5 runs, 2026-10-28 23:32:12 to 00:00:01). *Note: The pendant log is written with a delay: 5.3 s measured once (file size polled every second after a protective stop).*

Fault	How to provoke / revert	Logs and fault codes (observed)	Effect on signals (observed)	Category, sources and proof
controller_crash The controller process (URControl) is killed.	*Provoke:* docker exec ursim pkill -f URControl (SIGTERM) *Revert:* docker restart ursim (nothing restarts URControl inside the container)	**No fault code** controller WARNING (5/5): `Failed to write joint angles: [ #, #, #, #, #, #]` unknown INFO (5/5): `URControl::handler caught SIGTERM` controller INFO (5/5): `TCPServer: RTDE client disconnected` controller INFO (5/5): `WatchdogService::stopAllWatchdog()` controller INFO (5/5): `HW cache data was in sync` controller INFO (4/5): `URControl::shutdown`	Samples received: 58 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 1 cut(s), 15.0 s in total (median). No change of robot or safety mode. Dashboard after 15 s: NO_CONTROLLER / NORMAL. Black box: 0/5 runs. Our streams in error at the end: faults (5/5), signals (5/5), logs (5/5). Back to RUNNING: 5/5, 30.2 s (median).	**process** https://man7.org/linux/man-pages/man7/signal.7.html: SIGTERM, the default signal of pkill, asks a process to terminate. https://hub.docker.com/r/universalrobots/ursim_e-series: Official URSim e-Series Docker image. Evidence: `data/catalogue/controller_crash-*.json` (5 runs, 2026-10-28 23:32:38 to 00:00:27). *Note: The container's service directory (/home/root/service) is empty: no supervisor restarts URControl or PolyScope (read from /etc/service/*/run and /entrypoint.sh).*
freeze The whole controller is frozen for a few seconds (communication lost), then resumes.	*Provoke:* docker pause ursim, then docker unpause ursim after 8 s *Revert:* docker unpause ursim	**No fault code** No log line (besides connection noise).	Samples received: 227 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 1 cut(s), 9.0 s in total (median). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Back to RUNNING: 5/5, 1.1 s (median).	**process** https://docs.docker.com/reference/cli/docker/container/pause/: docker pause suspends all processes of a container. https://www.universal-robots.com/articles/ur/interface-communication/remote-control-via-tcpip/: e-Series Real-Time interface: 500 Hz by design. Evidence: `data/catalogue/freeze-*.json` (5 runs, 2026-10-28 23:33:34 to 00:01:22).
polyscope_freeze The PolyScope process (user interface, Dashboard server, pendant log) is frozen; the controller keeps running.	*Provoke:* docker exec ursim pkill -STOP -f felix.jar (PolyScope's Java process) *Revert:* docker exec ursim pkill -CONT -f felix.jar	**No fault code** controller WARNING (5/5): `Failed to send state to client #. Sending discontinued.`	Samples received: 481 per second over the 17 s window (median; 500 by design). No stream cut (0/5). No change of robot or safety mode. Dashboard after 15 s: unreachable (TimeoutError) / unreachable (TimeoutError). Black box: 0/5 runs. Back to RUNNING: 5/5, 1.1 s (median).	**process** https://man7.org/linux/man-pages/man7/signal.7.html: SIGSTOP stops a process, SIGCONT resumes it. https://www.universal-robots.com/articles/ur/interface-communication/overview-of-client-interfaces/: The Dashboard server (29999) sends commands to the GUI. Evidence: `data/catalogue/polyscope_freeze-*.json` (5 runs, 2026-10-28 23:34:00 to 00:01:48). *Note: Port 29999 is owned by PolyScope's Java process, ports 30001-30004 by URControl (netstat -ltnp in the container).* *Note: Killing PolyScope instead stops the whole container: its start script ends (observed once: 'container … is not running' 3 s after pkill).*

Fault	How to provoke / revert	Logs and fault codes (observed)	Effect on signals (observed)	Category, sources and proof
network_cut The container is disconnected from the network: every interface becomes unreachable.	*Provoke:* docker network disconnect bridge ursim *Revert:* docker network connect bridge ursim	**No fault code** polyscope ERROR (5/5): `ControllerSocket: # consecutive read timeouts, disconnecting` polyscope ERROR (5/5): `File attachment not found: /tmp/log/urcontrol/current` polyscope ERROR (4/5): `bdba6cff88 validated against root certificate 8761fffbf04ec4dd00da22eb78e0cbe7e62dc08f {thread: Thread-# , loggerClass: com.ur.polyscope.lic` polyscope WARNING (5/5): `Controller monitoring timed out too many times. Restarting connection` polyscope INFO (5/5): `Caused by: java.net.UnknownHostException: 89716e9d88ea: Temporary failure in name resolution` polyscope INFO (5/5): `#-#-# #:#:#,# pool-#-thread-# ERROR Could not determine local host name java.net.UnknownHostException: 89716e9d88ea: 89716e9d88ea: Temporary`	Samples received: 56 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 1 cut(s), 15.2 s in total (median). No change of robot or safety mode. Dashboard after 15 s: unreachable (ConnectionRefusedError) / unreachable (ConnectionRefusedError). Black box: 5/5 runs (SOFTWARE_FAILURE ×5). Our streams in error at the end: faults (5/5), signals (5/5). Back to RUNNING: 5/5, 5.2 s (median).	**network** https://docs.docker.com/reference/cli/docker/network/disconnect/: docker network disconnect removes a container from a network. Evidence: `data/catalogue/network_cut-*.json` (5 runs, 2026-10-28 23:34:36 to 00:02:24). *Note: Simulator side effect: inside the container PolyScope reaches the controller through the container's IP address, so their link breaks too.*
cpu_starvation The container is limited to 0.1 CPU: every process, the real-time controller included, is starved.	*Provoke:* docker update –cpus 0.1 ursim *Revert:* docker update –cpus <number of host CPUs> ursim (–cpus 0 does not remove a limit: observed)	**No fault code** No log line (besides connection noise).	Samples received: 153 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 6 cut(s), 0.9 s in total (median). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Back to RUNNING: 5/5, 1.1 s (median).	**resource** https://docs.docker.com/reference/cli/docker/container/update/: docker update –cpus limits the CPUs a running container may use. https://www.universal-robots.com/articles/ur/interface-communication/remote-control-via-tcpip/: "we cannot guarantee the frequency for primary, secondary and real-time client interfaces." Evidence: `data/catalogue/cpu_starvation-*.json` (5 runs, 2026-10-28 23:35:06 to 00:02:54).
cpu_overload 16 busy loops run inside the container, competing with the controller for the CPU.	*Provoke:* 16 shell busy loops in the container: sh -c 'while :; do :; done' *Revert:* docker exec ursim pkill -f 'while :; do :; done'	**No fault code** No log line (besides connection noise).	Samples received: 402 per second over the 17 s window (median; 500 by design). No stream cut (0/5). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Back to RUNNING: 5/5, 1.3 s (median).	**resource** https://www.universal-robots.com/articles/ur/interface-communication/remote-control-via-tcpip/: e-Series Real-Time interface: 500 Hz by design, frequency not guaranteed. Evidence: `data/catalogue/cpu_overload-*.json` (5 runs, 2026-10-28 23:35:44 to 00:03:35).

Fault	How to provoke / revert	Logs and fault codes (observed)	Effect on signals (observed)	Category, sources and proof
network_delay Every network packet of the container is delayed by 300 ms (± 50 ms).	*Provoke:* tc qdisc add dev eth0 root netem delay 300ms 50ms (helper container in URSim's network namespace) *Revert:* tc qdisc del dev eth0 root	**No fault code** controller WARNING (5/5): `Failed to send state to client #. Sending discontinued.`	Samples received: 71 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 29 cut(s), 12.8 s in total (median). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Our streams in error at the end: signals (4/5). Back to RUNNING: 5/5, 1.2 s (median).	**network** https://man7.org/linux/man-pages/man8/tc-netem.8.html: netem adds delay (and jitter) to outgoing packets. Evidence: `data/catalogue/network_delay-*.json` (5 runs, 2026-10-28 23:36:15 to 00:04:05).
packet_loss 20 % of the container's network packets are lost.	*Provoke:* tc qdisc add dev eth0 root netem loss 20% (helper container) *Revert:* tc qdisc del dev eth0 root	**No fault code** controller WARNING (5/5): `Failed to send state to client #. Sending discontinued.`	Samples received: 84 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 17 cut(s), 14.6 s in total (median). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Our streams in error at the end: signals (3/5). Back to RUNNING: 5/5, 1.2 s (median).	**network** https://man7.org/linux/man-pages/man8/tc-netem.8.html: netem drops a given percentage of packets. Evidence: `data/catalogue/packet_loss-*.json` (5 runs, 2026-10-28 23:36:47 to 00:04:33).
port_blocked Port 30003 (Real-Time interface, 500 Hz signals) is blocked; the other ports keep working.	*Provoke:* iptables -A INPUT -p tcp –dport 30003 -j DROP (helper container) *Revert:* iptables -D INPUT -p tcp –dport 30003 -j DROP	**No fault code** controller WARNING (5/5): `Failed to send state to client #. Sending discontinued.` polyscope WARNING (3/5): `Controller monitoring timed out too many times. Restarting connection`	Samples received: 60 per second over the 17 s window (median; 500 by design). Stream cuts in 5/5 runs: 1 cut(s), 15.0 s in total (median). No change of robot or safety mode. Dashboard after 15 s: RUNNING / NORMAL. Black box: 0/5 runs. Our streams in error at the end: signals (5/5). Back to RUNNING: 5/5, 1.2 s (median).	**network** https://man7.org/linux/man-pages/man8/iptables.8.html: An INPUT rule with target DROP discards incoming packets. https://www.universal-robots.com/articles/ur/interface-communication/overview-of-client-interfaces/: Port 30003 is the Real-Time interface. Evidence: `data/catalogue/port_blocked-*.json` (5 runs, 2026-10-28 23:37:15 to 00:05:05). *Note: Simulator side effect: PolyScope also uses port 30003 through the container's IP address, so its monitoring times out too.*

Fault	How to provoke / revert	Logs and fault codes (observed)	Effect on signals (observed)	Category, sources and proof
disk_full The disk holding URSim's files (logs, programs, black boxes) is filled up.	*Provoke:* dd if=/dev/zero bs=1M, then bs=4k, until the disk is full (URSim runs on its own 512 MB disk) *Revert:* rm the fill files	**C209A0** (5/5 runs, after 2391 ms) — *caused by the probe: protective_stop() sent 2 s after the injection* *Official: C209 A protective stop was triggered (for test purposes only)* polyscope ERROR (5/5): `java.io.IOException: No space left on device` polyscope ERROR (4/5): `Could not close FlightRecord zip file` polyscope ERROR (3/5): `File attachment not found: /tmp/log/urcontrol/current` pendant WARNING (5/5): `C209A0` pendant INFO (5/5): `C0A0 protective_stop Program protective_stop stopped` pendant INFO (5/5): `C0A0 protective_stop Program protective_stop started`	Samples received: 481 per second over the 17 s window (median; 500 by design). No stream cut (0/5). safety_mode: PROTECTIVE_STOP (5/5) Dashboard after 15 s: RUNNING / PROTECTIVE_STOP. Black box: 5/5 runs (unreadable zip (331776 bytes) ×3, unreadable zip (0 bytes) ×1, unreadable zip (327680 bytes) ×1). Back to RUNNING: 5/5, 1.1 s (median).	**resource** https://man7.org/linux/man-pages/man1/dd.1.html: dd copies data until the output fails (here: no space left on device). https://www.universal-robots.com/manuals/EN/HTML/SW5_26/Content/prod-err-codes/topics/CODE_209.html: official page of the observed code. Evidence: `data/catalogue/disk_full-*.json` (5 runs, 2026-10-28 23:37:48 to 00:05:37). *Note: Silent while the robot is idle: the captures send protective_stop() 2 s after the fill to force the controller to write.* *Note: With 1 MB blocks only, up to 1 MB stays free: a 329 kB black box was still written (observed once); hence the 4 kB second pass.*